Assignment 1 - Git & GitHub Fundamentals

Report

Full Name: Iman El Isamy
Student ID: 157195
Course Code & Section: CSC 3324 - Section 02
GitHub Username: elisamyiman8-hub
Repository: https://github.com/elisamyiman8-hub/git-session1-elisamyiman8-hub

1. Repository Setup

I unzipped the files. Then I renamed the folder to git-session1-elisamyiman8-hub. After that I ran git init to set up the repository.

My.gitignore file ignores local.env and any files ending with.log.. I added !important.log so that important.log is not ignored and Git can track it.

I checked everything to be sure. I used git check-ignore -v git status --ignored. Git ls-files. All three commands showed the thing: local.env and two.log files are ignored and important.log is tracked. It all matches what I expected. Everything looks good.

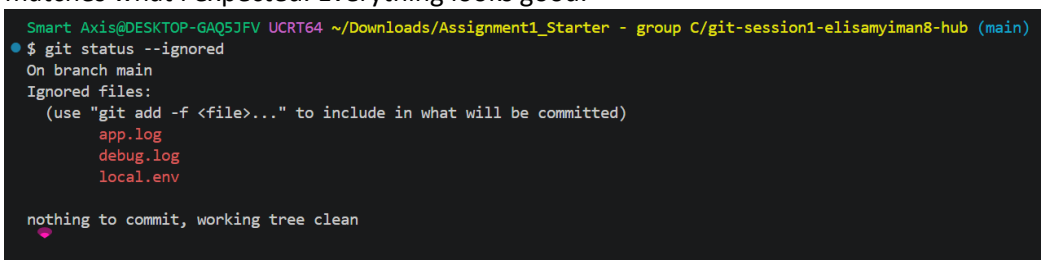


Figure 1 - `git status --ignored`: app.log, debug.log and local.env are ignored.



Figure 2 - `git check-ignore -v`: each ignored file and the .gitignore rule that matches it.



Figure 3 - `git ls-files`: important.log is tracked; local.env and the other logs are not.

2. README & Commit History

My README.md has my name, my GitHub username, a description, the technologies that I used a code block, a link and the project banner that was provided. I created the history using logical commits. The commits were setup and documentation homepage text, homepage styling, notes and app changes the About feature and the final README/notes updates. I worked on README.md, index.html style.css, app.js

and notes.txt. The log below shows 9 commits at that time; one more commit was added later for the diff demo. There were 10 commits, in total.

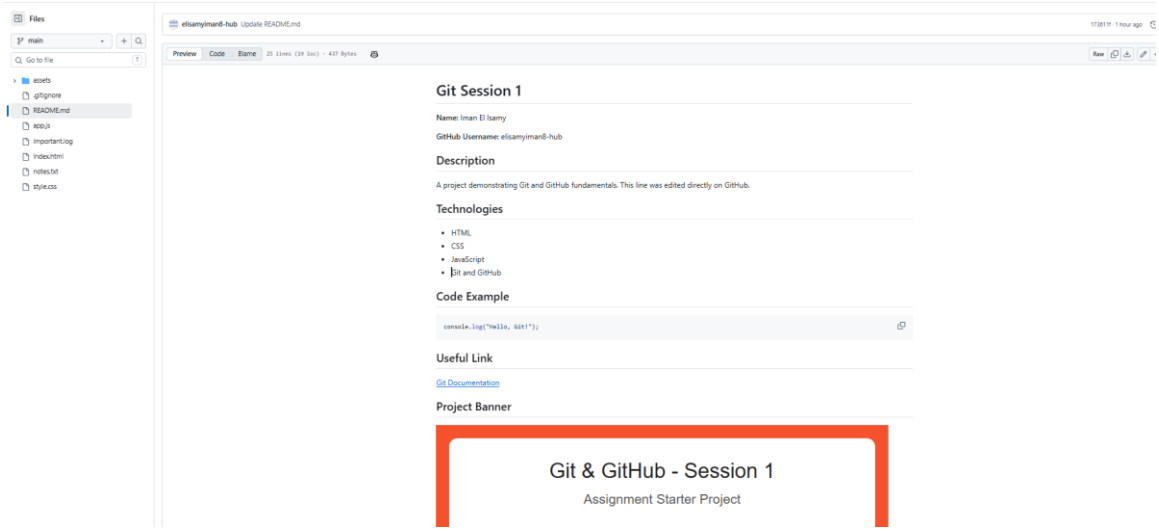


Figure 4 - Rendered README on GitHub (after my edit made on GitHub).

```
Smart Axis@DESKTOP-GAQ5JFV UCRT64 ~/Downloads/Assignment1_Starter - group C/git-session1-elisamyiman8-hub (main)
$ git check-ignore -v local.env app.log debug.log
.gitignore:1:local.env  local.env
.gitignore:2:*.log      app.log
.gitignore:2:*.log      debug.log

Smart Axis@DESKTOP-GAQ5JFV UCRT64 ~/Downloads/Assignment1_Starter - group C/git-session1-elisamyiman8-hub (main)
$ git log --all --oneline
b7862a4 (HEAD -> main, origin/main, origin/HEAD) Add notes about workflow
173611f Update README.md
6e9a1cd Log message when app loads
99ce66d (about-feature) Add About section
8f63af7 Style main heading
9554d90 Update app and notes
26d4ef7 Improve homepage styling
68ab564 Improve project homepage introduction
faadd25 Initial project setup and documentation
```

Figure 5 - Compact history with `git log --all --oneline`; main and origin/main are in sync.

3. Selective Staging

I edited `style.css` and `app.js` at the same time. `git status` showed both as modified; I ran `git add style.css` only, so `style.css` was under 'Changes to be committed' and `app.js` under 'Changes not staged'. I committed only `style.css` ('Style main heading'), and `git status` then showed `app.js` still modified. I committed `app.js` later as a separate commit.

```
Smart Axis@DESKTOP-GAQ5JFV UCRT64 ~/Downloads/Assignment1_Starter - group C/git-session1-elisamyiman8-hub (main)
$ git status
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   style.css

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js
```

Figure 6 - One file staged (style.css), one left unstaged (app.js).

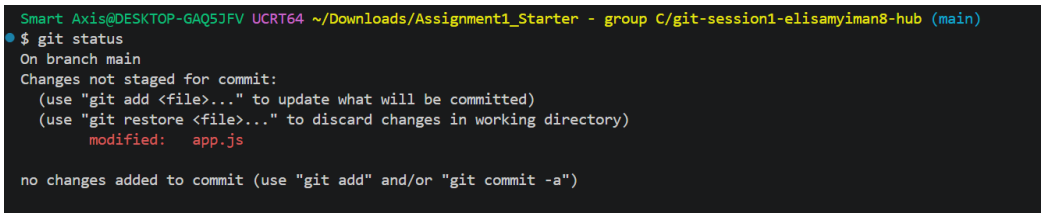


Figure 7 - After committing style.css, app.js is still modified.

4. Branch & Merge

I created the `about-feature` branch with `git branch` and `git switch`, added an About section to `index.html` and committed it there. I then switched back to `main` and ran `git merge about-feature`. Because `main` had not moved in the meantime, this was a fast-forward merge, so the history stays a straight line with the About commit now part of `main`.

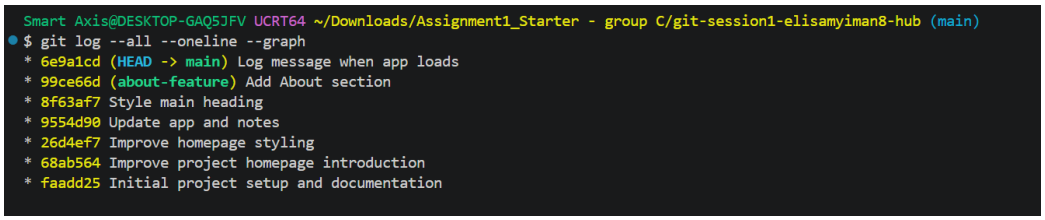


Figure 8 - `git log --all --oneline --graph`: the About commit on about-feature is included in main.

5. Remote Changes

A **commit** records staged changes in my *local* repository history. A **push** uploads those local commits to the remote repository on GitHub. I connected GitHub with `git remote add origin ...` and published `main` with `git push -u origin main`. The first push failed with a temporary GitHub 'Internal Server Error'; I ran the same command again and it succeeded.

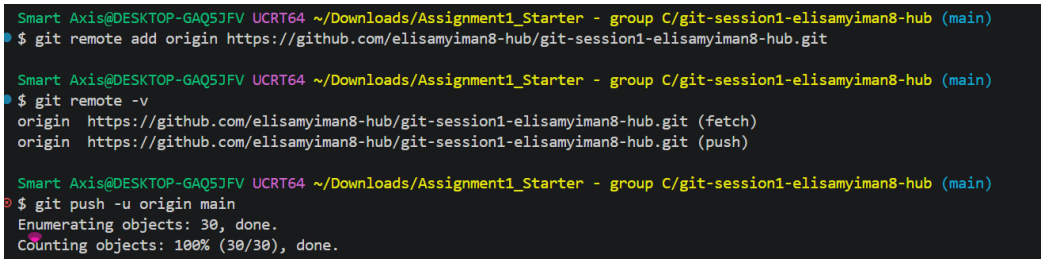


Figure 9 - Remote added and verified with `git remote -v`, then the push started.

Fetch vs. pull: I edited README.md directly on GitHub. `git fetch` downloaded the new commit and updated `origin/main` without changing my files, and `git status` said my branch was behind by 1 commit. `git pull origin main` then fetched and merged that commit into my local `main` (a fast-forward).

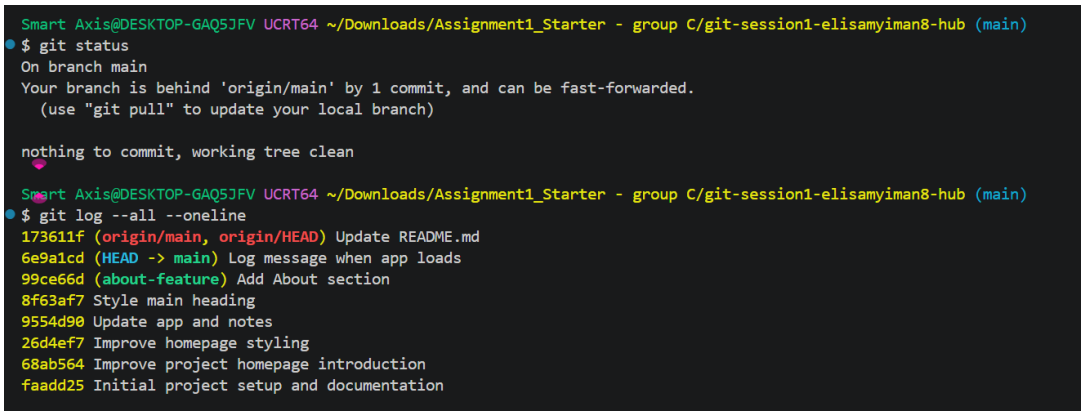


Figure 10 - After fetch: local main is behind origin/main by 1 commit, and the remote commit appears in the log.

I also inspected my changes before committing `git diff` shows an unstaged change, and `git diff --staged` shows what is staged for the next commit.

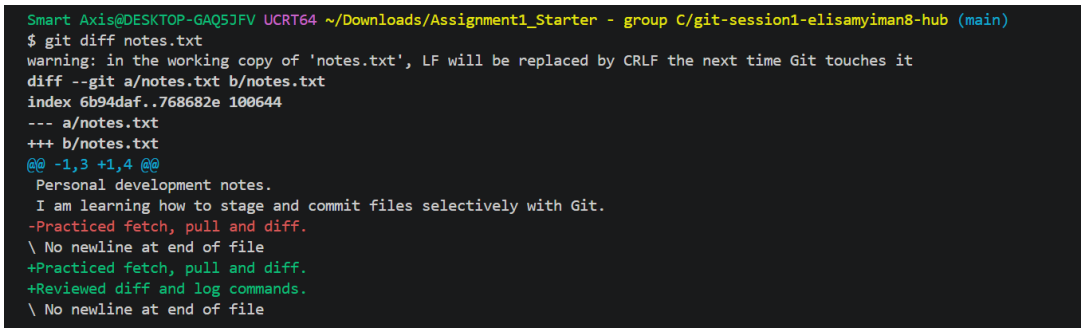


Figure 11 - Unstaged diff of notes.txt (`git diff notes.txt`).

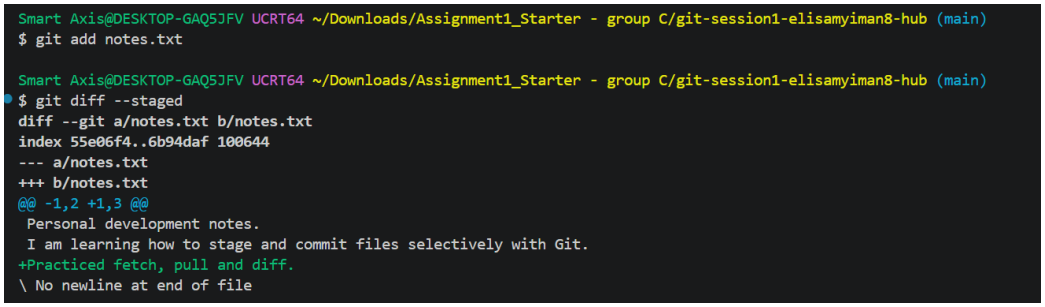


Figure 12 - Staged diff of notes.txt (`git diff --staged`).

6. Fair Use of AI

I used an AI assistant called Claude as a guide while completing this lab. I supplied the AI assistant the assignment PDF, the files and the Session 1 slides. I asked the AI assistant for a step-by-step plan for explanations of each command and for help with interpreting the output that I received. For example I wanted to understand the 'behind origin/main' message, the LF/CRLF warning and the GitHub 'Internal Server Error' that appeared on my push. performed all file edits myself including changes to the README, homepage text, styling and the About section. I also chose my own commit messages.I verified every step

by running each command myself and checking that the output matched the description in the slides and the assignment.

7. Sources & Reflection

Sources: Session 1 course slides (Git Basics & Your First GitHub Repository); the Assignment 1 instructions and starter files; Atlassian Git tutorial on. gitignore (https://www.atlassian.com/git/tutorials/saving-changes/gitignore).

Reflection: I find the concept to be the difference between the working directory, the staging area and the commit history. Git status shows all three at once which confused me until I saw one staged and another not; then selective staging finally made sense. Fetch versus pull also confused me until I realised that fetch updates origin/main and leaves my files unchanged. When Git behaves unexpectedly, I first run git status to see the branch what is. Whether I am ahead or behind the remote. Then I run git log --all, --oneline to review the history.